

Homelab Overview: Proxmox Host and Project Hub

Documented October 2026 (originally July 2026)

Intent for this project

I built this homelab to have a real, hands-on environment for cybersecurity and IT administration practice, rather than just studying concepts in isolation. It doubles as a portfolio piece since employers in defensive security and sysadmin roles care a lot about seeing actual infrastructure work, not just certifications or coursework. It's also a genuinely good project for learning, honing skills from University, and experimenting with avenues I enjoy.

This project started July 4, 2026, and is actively evolving. This document is the high-level overview. Each major piece has its own write-up, listed below.

Related write-ups: Homelab DNS Stack (Pi-hole + Unbound), Homelab Network Segmentation: pfSense, and Power Optimization: Idle Draw Baseline.

Why Proxmox

I chose Proxmox VE over alternatives like ESXi or running everything in Docker on bare Linux for two main reasons. First, it is free and open source, so there is no licensing cost or restriction getting in the way of experimenting. Second, it gives me real flexibility between full VMs and lightweight LXC containers on the same host, which mirrors the kind of virtualization environment I would expect to work with in an actual IT admin or sysadmin role. Learning Proxmox specifically is also directly relevant experience for that kind of job. Furthermore, prior academic experience with Proxmox allowed for an accelerated deployment timeline.

Why not use dedicated hardware?

The machine hosting all of this is a repurposed office PC rather than something bought specifically for homelabbing. Two reasons factored into that decision. First, it kept the cost of the project at basically zero since the hardware was already sitting unused. Second, it forces me to actually think about resource constraints, since I am working with 16GB of RAM total rather than an unlimited budget of compute. Having to plan around that, deciding what runs full time versus on demand, is its own kind of practical systems administration skill.

Total hardware cost: \$0. The machine was already sitting unused, repurposed instead of bought.

Physical Setup

The machine sits next to my main desktop. It runs headless, no monitor, mouse, or keyboard plugged in,

since everything is managed through the Proxmox web UI over the local network.

Hardware specifications

CPU	Intel Core i5-6600 @ 3.30GHz, 4 cores, 1 socket
RAM	15.54 GiB total
Storage	1TB external SSD
Swap	8.00 GiB
Network interfaces	Onboard NIC (pfSense WAN) + USB RTL8153 (pfSense LAN)
Boot mode	EFI
Hypervisor	Proxmox VE, pve-manager 9.2.2
Kernel	Linux 7.0.2-6-pve

Snapshot (October 2026, with only the pfSense VM running): 1% CPU across 4 cores, 24% RAM (3.67 GiB of 15.50 GiB), and 2% of the 888 GiB storage pool. Heavier guests like Wazuh run on demand rather than 24/7. The host has an onboard NIC (pfSense WAN side) and a USB RTL8153 adapter (pfSense LAN side) .

Network Architecture

There is one Ethernet drop from the wall and no modem, so only one device can be “the thing plugged into the wall.” The TP-Link Archer stays the household router on 192.168.0.0/24 (Wi-Fi, TVs, and the Proxmox host’s own management IP). pfSense sits behind it in a double-NAT arrangement: its WAN side takes a normal DHCP lease from the Archer through the onboard NIC, and its LAN side (192.168.10.0/24) runs through a USB RTL8153 on a separate Proxmox bridge into a 5-port unmanaged switch. Only devices I deliberately plug into that switch are behind pfSense. Their DNS goes to pfSense, which forwards to Pi-hole/Unbound.

I originally considered making pfSense the router for the whole apartment. I backed away from that because pfSense is a VM on a host that isn’t dedicated hardware, so rebooting or powering off the host would take down the whole household’s internet.

Proxmox environment overview

The host defines six guests, started on demand to stay within the 16GB RAM ceiling:

- LXC 100, pihole (Debian 12): Pi-hole + Unbound. Full write-up: Homelab DNS Stack.
- LXC 101, tailscale-router: collaborator access, described below.

- LXC 201, spiderfoot: General OSINT Personal education tool
- VM 102, pfsense (2 vCPU, 2GB RAM, 16GB disk): Full write-up: Homelab Network Segmentation: pfSense.
- VM 103, metasploitable2: intentionally vulnerable practice target
- VM 104, flowsint: Untouched as of October

Everything shares one storage pool on the 1TB external SSD. Tracking RAM and storage allocation matters because the 16GB ceiling is a real constraint.

Security Posture

The Proxmox host itself hasn't been hardened yet. Planned steps include restricting web UI access to LAN only and enabling 2FA on the root account. The pfSense VM has had a light hardening pass (WebGUI unreachable from WAN, SSH disabled, login protection on, updated to 2.8.1). A fuller rule-tightening pass will come once the segment faces a real internet-facing threat model.

Power

After the pfSense deployment I measured idle draw using turbostat and PowerTOP. CPU package power was about 11.1 W with the GTX 960 installed, about 4.7 W with it removed, and about 1.1 W with the pfSense VM also shut down. These are CPU package figures, not wall power. The package never reaches deep sleep states in any configuration, and the cause is still being investigated. Full details: Power Optimization: Idle Draw Baseline.

Planned next steps

- Harden the Proxmox host (restrict web UI, enable 2FA)
- pfSense: scoped firewall rules, more devices on the switch (Raspberry Pi next), VLANs and Suricata/Snort evaluation
- Power: package C-state investigation, wall meter reading, pfSense VM wakeup reduction
- Wazuh SIEM
- Kali Linux VM
- Jellyfin with GPU passthrough (GTX 960, currently removed)
- Proxmox-level scoped users and resource pools
- Possible future project: dedicated always-on low-power hardware for pfSense

Remote collaboration access

As collaborators join the project, network-level access needs to be granted without exposing the home network broadly or handing out direct credentials to internal infrastructure.

The approach used is a Tailscale subnet router, deployed as a dedicated lightweight LXC container rather than installed directly on the Proxmox host. This keeps the hypervisor itself untouched by the remote access layer and isolates that role to a single-purpose container.

Rather than granting full LAN access, collaborator devices are tagged and restricted through Tailscale's access control policy to only the specific services they need, scoped by IP and port rather than by subnet. Everything else on the network remains unreachable to a tagged device, regardless of what the underlying subnet route technically covers, the ACL is the actual enforcement boundary, not the route itself.

This is intentionally a software-level boundary. pfSense now exists, but it runs as an isolated segment rather than the household router, so the ACL remains the real enforcement boundary for collaborator access.

Proxmox-level permissions (scoped users, resource pools) are the next layer planned, so collaborator access to specific VMs/containers can be similarly restricted at the hypervisor level, not just the network level.

Screenshots



